Домашнее задание: `Docker` и `CI/CD` для API-автотестов

Техническая информация

Параметр	Значение
Дисциплина	Тестирование ПО · ИТМО · осень 2026
Занятие	6 — «Интеграция тестов в CI/CD»
Проект	CI в тестировани
Стек	Docker · Git · GitHub Actions / GitLab CI
Трудоёмкость	3-5 часов
Количество попыток	1
Оценивание	до 10 баллов + 2 дополнительных
Проверяющий	преподаватель
Формат оценивания	по критериям
Дедлайн	20.10
Разбор задания	20.10

* Максимум по основным критериям — **10 баллов**. Ещё **2 дополнительных балла** можно получить за запуск собранного контейнера с тестом непосредственно в CI pipeline.

Цель задания

Научиться упаковывать уже существующие API-автотесты в Docker контейнер и подключать их к CI: собирать воспроизводимый Docker image из репозитория, запускать тест в контейнере и настраивать автоматическую сборку в GitHub Actions или GitLab CI.

Ваш результат

В результате выполнения задания вы:

- возьмёте уже существующий API-автотест и подготовите его к запуску внутри контейнера;
- создадите `Dockerfile` и `.dockerignore`;
- соберёте Docker image и убедитесь, что при `docker run` тест запускается автоматически;
- настроите CI pipeline, который автоматически собирает Docker image из содержимого Git-репозитория;
- оформите README с командами локального запуска и ссылкой на успешный pipeline.

Материалы для выполнения

Для работы используйте:

- любой API-автотест, который уже работает локально и обращается к общедоступному API;
- `Docker` или совместимую среду для сборки и запуска контейнеров;
- `GitHub Actions` или `GitLab CI`;
- репозиторий с исходниками теста.

Подготовка

Новый API-тест в рамках этого задания писать не требуется. Используйте тест из предыдущей работы, семинара или собственный уже существующий тест.

- До контейнеризации убедитесь, что тест запускается локально стандартной командой проекта и действительно выполняет HTTP-запрос.
- Тест должен завершаться успешным exit code при прохождении проверок и ненулевым exit code при падении.
- Репозиторий должен быть доступен CI-системе.

Задача

Ваша задача — превратить существующий API-тест в воспроизводимый контейнерный запуск и настроить автоматическую сборку этого контейнера в CI.

Ключевая цепочка задания: `repository → docker build → image → docker run → test result`.

Что нужно сделать

1. Запакуйте существующий API-тест в Docker

Создайте Dockerfile, который содержит всё необходимое для запуска теста в изолированном окружении.

- выберите подходящий базовый image;
- скопируйте в image файлы проекта, необходимые для запуска теста;
- установите зависимости проекта;
- задайте команду, которая запускает тест при старте контейнера;
- добавьте `.dockerignore` и исключите из build context ненужные локальные файлы.

Пример команд проверки:

```
docker build -t api-test .
docker run --rm api-test
```

2. Проверьте поведение контейнера

При запуске контейнера тест должен выполняться автоматически, без входа в контейнер и без ручного запуска команды тестирования.

- контейнер обращается к API;
- в консоли виден результат выполнения теста;
- при успешном тесте процесс завершается с `exit code 0`;

- при падении теста процесс завершается с ненулевым exit code.

3. Настройте CI pipeline для сборки Docker image

Создайте CI-конфигурацию для выбранной платформы:

- `GitHub Actions`: `.github/workflows/ci.yml`;
- `GitLab CI`: `.gitlab-ci.yml`.

Pipeline должен автоматически запускаться как минимум при push и выполнять корректную сборку Docker image из текущего состояния репозитория.

- получить исходный код репозитория;
- выполнить `docker build`;
- завершиться успешно при корректной сборке и упасть, если image собрать невозможно.

4. Оформите README

README.md должен позволять другому человеку воспроизвести запуск без дополнительных устных инструкций.

- кратко укажите, какой API-тест используется;
- добавьте команду сборки Docker image;
- добавьте команду запуска контейнера;
- укажите, где находится CI-конфигурация и при каком событии запускается pipeline.

Дополнительное задание (+2 балла)

После успешной сборки image запустите этот же image непосредственно внутри CI pipeline.

Pipeline должен реализовывать цепочку: `push → docker build → docker run → API test → pipeline result`.

- если тест внутри контейнера проходит, job завершается успешно;
- если тест внутри контейнера падает, job и pipeline должны стать неуспешными.

```
docker run --rm api-test
```

Важные ограничения

- Не требуется писать новый API-тест специально для этой работы. Оценивается контейнеризация и CI-интеграция.
- Не запускайте тест вручную после старта контейнера: команда тестирования должна быть частью образа/контейнерного запуска.
- Не используйте локальные абсолютные пути и файлы, которых нет в репозитории.
- Pipeline должен собираться из чистого состояния репозитория и не зависеть от заранее подготовленного локального image.
- Секреты и токены, если они нужны выбранному API, нельзя хранить в репозитории. Используйте CI secrets/variables.

Критерии проверки

Задание оценивается по следующим критериям. Максимум по основным критериям — 10 баллов.

Критерий	2 балла	1 балл	0 баллов
Dockerfile и image	Dockerfile корректен; image собирается из чистого checkout; зависимости устанавливаются внутри image.	Image собирается, но есть ручные шаги или зависимость от локальной среды.	Dockerfile отсутствует или image не собирается.
Запуск теста в контейнере	Ручное выполнение docker run запускает API-тест; логи видны; exit code отражает успех/падение.	Тест запускается, но требуется дополнительное действие или exit code/логи настроены неполно.	Контейнер не запускает тест либо запуск неработоспособен.
CI-конфигурация	Pipeline корректен, запускается при push и содержит понятные воспроизводимые шаги.	Pipeline запускается, но триггер, структура или настройки частично некорректны.	CI-конфигурация отсутствует или pipeline не запускается.
Сборка Docker image в CI	CI выполняет docker build из репозитория; ошибка сборки приводит к падению job.	Сборка работает, но зависит от ручной подготовки runner или готового image.	Docker image в CI не собирается.
Воспроизводимость и оформление	Есть .dockerignore и README; команды воспроизводимы; нет зависимости от IDE и локальных путей.	README, .dockerignore или инструкции присутствуют не полностью.	Проект требует ручной доработки перед запуском.
Итого			**10 баллов**

Дополнительные баллы (+2): pipeline запускает собранный Docker image с API-тестом, а exit code теста напрямую определяет статус job/pipeline.

Что и как сдавать

Отправьте:

1. ссылку на Git-репозиторий с исходниками теста, `Dockerfile`, `.dockerignore` и CI-конфигурацией;
2. `README.md` с командами сборки и запуска контейнера;
3. ссылку на успешно выполненный pipeline.

Формат сдачи: **ссылка на репозиторий / ветку**.
Дедлайн: **20.10**

Команды

Минимальная локальная проверка:

```
docker build -t api-test .
docker run --rm api-test
```